

EcoModelo

Calibración de una Sucesión Poblacional mediante Backtracking

Juan Sebastian Yepez
Sebastian Pardo
Simon Ospina Gomez

Universidad Nacional de Colombia - Sede Manizales

Introducción a las Ciencias de la Computación

Profesor: Carlos Manuel Orrego

19 de junio de 2026

1. Comprensión del Problema

El Desafío

Calibrar un modelo matemático para que sus predicciones se ajusten a la realidad observada.

Datos Reales Observados:

[10, 12, 15, 19, 24, 30]

Modelo Logístico Discreto

$$x(t+1) = x(t) + r \cdot x(t) \cdot \left(1 - \frac{x(t)}{K}\right)$$

Debemos hallar los mejores parámetros para:

- r : Tasa de crecimiento.
- K : Capacidad de carga.

2. Modelamiento con Backtracking

En lugar de usar fuerza bruta ciega, modelamos el problema como un **árbol de decisiones**.

Espacio de Búsqueda

- $r \in \{0,1,0,2,0,3,0,4\}$
- $K \in \{50,80,100,120\}$

Total de combinaciones: 16 escenarios base.

Solución Parcial

Es la secuencia de población simulada paso a paso (día a día), junto con su **error absoluto acumulado**.

3. Restricciones y Poda (Retroceso)

El algoritmo abandona un camino (*backtrack*) si la solución parcial actual viola alguna de estas dos reglas:

1. Restricción Física (Valores Absurdos)

Si la fórmula arroja $x(t + 1) < 0$.

Motivo: Una población negativa es biológicamente imposible.

2. Restricción de Eficiencia (Error Alto)

Si `error_acumulado` $\geq$ `mejor_error_global`.

Motivo: La rama actual ya es peor que nuestro mejor récord. Se corta la simulación para ahorrar procesamiento.

4. Ejemplo de Poda Manual

Supongamos la combinación $r = 0,4$, $K = 50$.

Récord actual a vencer: `mejor_error_global` = 5.0

Día	Simulación	Dato Real	Error Acumulado
Día 0	10.00	10	0.00
Día 1	13.20	12	1.20
Día 2	17.08	15	3.28
Día 3	21.57	19	5.85

Acción del Algoritmo: Como $5,85 > 5,0$, se activa el *break*. Se realiza la **PODA** y no se calculan los días 4 y 5.

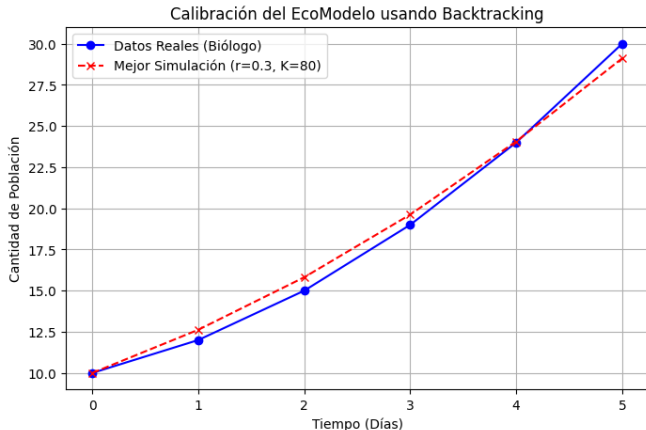
5. Resultados de la Calibración

Mejores Parámetros

- r óptimo: [0.3]
- K óptimo: [80]
- Error total: [3.01]

Eficiencia

Ahorro computacional frente a la fuerza bruta: **[18.75] %**



6. Análisis de Complejidad (Notación $\mathcal{O}$ Grande)

La Notación de Landau ($\mathcal{O}$ Grande) mide qué tan pesado se vuelve un programa al escalar los datos.

El “Peor de los Casos” (Fuerza Bruta)

Ocurre si el algoritmo evalúa todas las tasas (N), todas las capacidades (M) en todos los días (T). La complejidad matemática es: $\mathcal{O}(N \times M \times T)$

¿Qué aprendimos con este proyecto?

- **No conformarnos:** En la práctica, no tenemos que sufrir el tiempo dictado por el “peor escenario” teórico.
- **Algoritmo inteligente:** Al implementar la poda, el código detecta combinaciones inviables y corta el camino tempranamente.
- **Eficiencia:** Evitamos múltiples iteraciones inútiles, logrando que el programa sea mucho más rápido que la fuerza bruta.

7. Implementación de la Poda en el Código

Así se ve reflejada la lógica matemática en nuestra programación:

Fragmento del Algoritmo (Corte de Ejecución)

```
# Calculamos el error de este día
error_del_paso = abs(siguiete_poblacion - datos_reales[t])
error_acumulado = error_acumulado + error_del_paso

# --- RESTRICCIÓN 2: PODA POR ERROR ALTO ---
if error_acumulado >= mejor_error_global:
    hubo_poda = True

    # AQUÍ OCURRE EL BACKTRACKING
    break
```

Nota: El comando `break` es la instrucción crucial que interrumpe el ciclo `for` temporal.

8. Conclusiones

- **Exploración Inteligente:** El backtracking demostró ser superior a la fuerza bruta al descartar ramas ineficientes desde sus etapas tempranas.
- **Independencia de estados:** Superamos un error de modelamiento inicial asegurándonos de resetear las variables de control (como el error acumulado) al inicio de cada iteración del árbol de búsqueda.
- **Visión Computacional:** Entendimos que optimizar no solo se trata de procesar más rápido, sino de saber matemáticamente **qué no es necesario calcular**.

¡Muchas gracias por su atención!